Topic: Types of Queue Circular Queue, Priority Queue

Q. Explain the limitations of a simple linear queue. How do a Circular Queue and a Priority Queue overcome specific challenges? Describe their working principles.

> **Definition to Remember**

> A **
**Prior
tasks t

Queue Types: Circular & Priority Queues

In a standard linear array queue, once the `rear` pointer reaches the maximum index (`SIZE - 1`), no new elements can be enqueued. This happens **even if spaces are empty at the front** due to earlier dequeue operations, leading to severe **memory wastage**.

2. Circular Queue

Solution: The queue forms a logical circle. The next position after the last index is index 0.

- **Working Principle:** Uses the **modulo operator (`%`)** to wrap pointers around.

- Nex

```
```

[0] <--

3. Priority Queue

Problem: A standard queue strictly processes the oldest element first. Some real-world tasks (like CPU interrupts) are urgent and cannot wait in a FIFO line.

**Solu

- **Working Principle:**

- Ele

> **Must-Write Points (for 10 marks)**

> 1. **

> **Quick Recall**

> `Line
by high

Complete Executable C Code: Circular Queue Array Implementation

```c
#includ

#define MAX 5

struct CircularQueue {
    int iten

void initQueue(struct CircularQueue *q) {
    q->fron

int isFull(struct CircularQueue *q) {
    return

int isEmpty(struct CircularQueue *q) {
    return

void enqueue(struct CircularQueue *q, int val) {
    if (isFu
```

```
int dequeue(struct CircularQueue *q) {
                                                                    if (isEm
```

```
void display(struct CircularQueue *q) {
                                                                    if (isEm
```

```
int main() {
                                                                    struct C
enqueue(&q, 10);
                                                                    enqueu
printf("Dequeued: %d\n", dequeue(&q));
                                                                    printf("
enqueue(&q, 50);
                                                                    enqueu
return 0;
                                                                    }
```